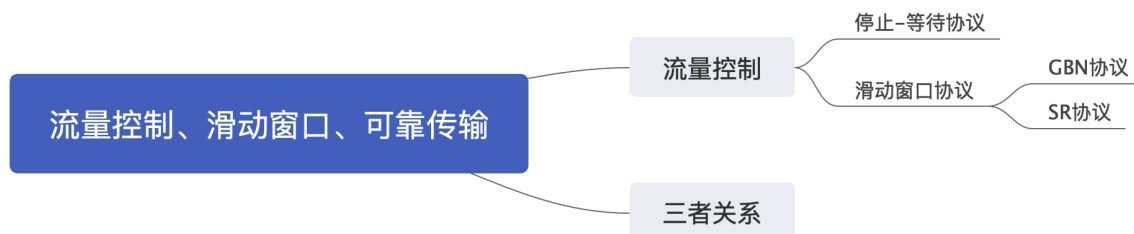


第三章 数据链路层 3.4 流量控制、可靠传输与滑动窗口机制

3.4.1 流量控制、可靠传输与滑动窗口机制



流量控制涉及对链路上的帧的发送速率的控制，以使接收方有足够的缓冲空间来接受每个帧。

例如，在面向帧的自动重传请求系统中，当待确认帧的数量增加时，有可能超出缓冲存储空间而造成过载。

流量控制的基本方法是由接收方控制发送方发送数据的速率。
常见的方式有两种：

- 停止-等待流量控制；
- 滑动窗口流量控制；

停止-等待协议

发送窗口大小=1，接收窗口大小=1；

后退N帧协议（GBN）

发送窗口大小>1，接收窗口大小=1；

选择重传协议（SR）

发送窗口大小>1，接收窗口大小>1；

滑动窗口协议分为（1）后退N帧协议（GBN）（2）选择重传协议（SR）

停止-等待流量控制基本原理：

发送方每发送一帧，都要等待接收方的应答信号，之后才能发送下一帧；

接收方每接受一帧，都要反馈一个应答信号，表示可接受下一帧；

如果接收方不反馈应答信号，那么发送方必须一直等待。

每次只允许发送一帧，然后就陷入等待接收方确认信息的过程中，因而传输效率很低。

滑动窗口流量控制基本原理：

在任意时刻，发送方都维持一组连续的允许发送的帧的序号，称为发送窗口；

接收方也维持一组连续的允许接受帧的序号，称为接受窗口。

发送窗口用来对发送方进行流量控制，而发送窗口的大小 W ，代表在还未收到对方确认信息的情况下发送方最多还可以发送多少个数据帧。

同理，在接收端设置接受窗口是为了控制可以接受哪些数据帧和不可以接受哪些帧。

在接收方，只有收到的数据帧的序号落在接受窗口内时，才允许将该数据帧收下。若接收到的数据帧落在接受窗口之外，则一律将其丢弃。

发送端每收到一个确定帧，发送窗口就向前滑动一个帧的位置，当发送窗口内没有可以发送的帧（即窗口内的帧全部是已发送但未收到确认的帧）时，发送方就会停止发送，直到收到接收方发送的确认帧使窗口移动，窗口内有可以发送的帧后，才开始继续发送。

接收端收到数据帧后，将窗口向前移一个位置，并发回确认帧，若收到的数据帧落在接受窗口之外，则一律丢弃。

图 3.7 给出了发送窗口的工作原理，图 3.8 给出了接收窗口的工作原理。

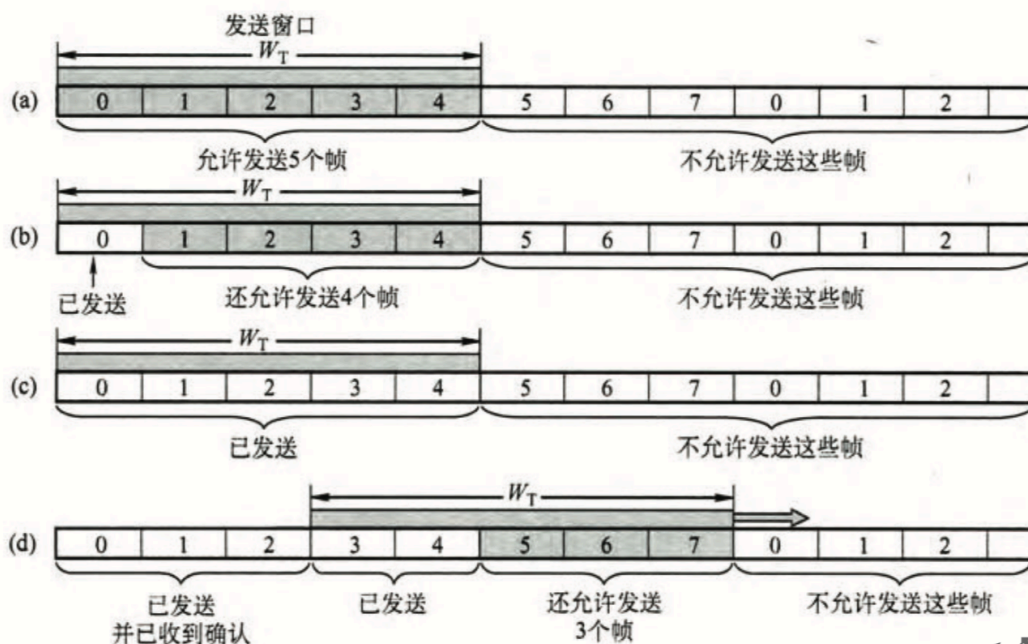


图 3.7 发送窗口控制发送端的发送速率：(a)允许发送 0~4 号共 5 个帧；(b)允许发送 1~4 号共 4 个帧；(c)不允许发送任何帧；(d)允许发送 5~7 号共 3 个帧

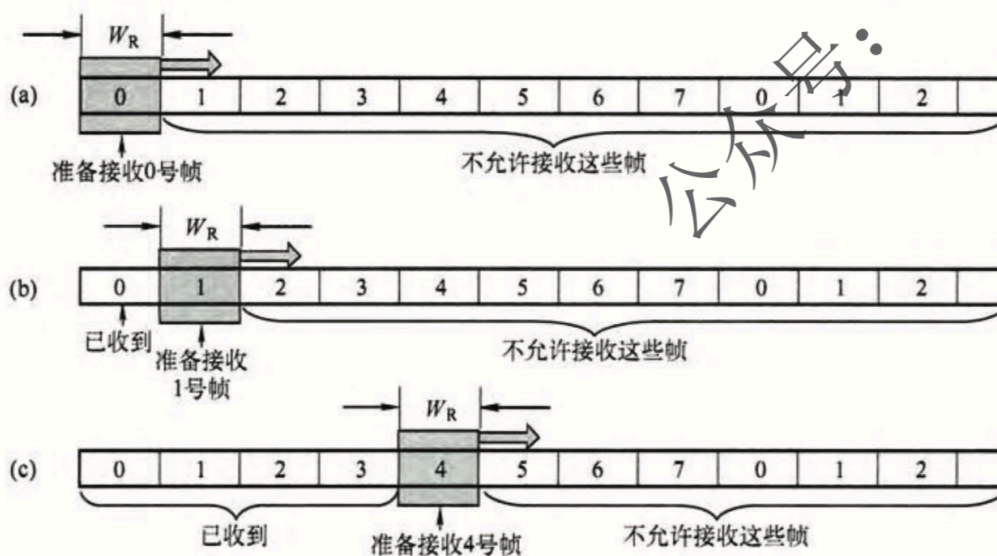


图 3.8 $W_R = 1$ 的接收窗口的意义：(a)准备接收 0 号帧；(b)准备接收 1 号帧；(c)准备接收 4 号帧

滑动窗口有以下重要特性：

(1) 只有接受窗口向前滑动（同时接收方发送了确认帧）时，发送窗口才有可能（只有发送方收到确认帧后才一定）向前滑动。

(2) 从滑动窗口的概念看，停止-等待协议，后退N帧协议和选择重传协议只有发送窗口大小与接受窗口大小上有所差别：

停止-等待协议：发送窗口大小=1，接受窗口大小=1。

后退N帧协议：发送窗口大小>1，接受窗口大小=1。

选择重传协议：发送窗口大小>1，接受窗口大小>1。

(3) 接受窗口大小为1时，可保证帧的有序接受。

(4) **数据链路层的滑动窗口协议中，窗口的大小在传输过程中是固定的。**

可靠传输机制：

数据链路层的可靠传输通常使用**确认**和**超时重传**两种机制来完成。

确认是一种**无数据的控制帧**，这种控制帧使得接收方可以让发送方知道哪些内容被正确接收。

有些情况下为了提高传输效率，**将确认捎带在一个回复帧中**，称为**捎带确认**。

超时重传是指发送方在发送某个数据帧后就开启一个**计时器**，**在一定时间内如果没有得到发送的数据帧的确认帧**，那么就重新发送该数据帧，知道发送成功为止。

自动重传请求 (ARQ) 通过接收方请求发送方重传出错的数据帧来恢复出错的帧，是通信中用于处理信道所带来的差错的方法之一。

传统自动重传请求分为三种，即

(1) **停止-等待ARQ** (2) **后退N帧ARQ** (3) **选择性ARQ**

后退N帧ARQ和选择性ARQ是滑动窗口技术与请求重发技术的结合。由于窗口尺寸开到足够大时，帧在线路上可以连续地流动，因此又称其为**连续ARQ协议**。

注意：在数据链路层中流量控制机制和可靠传输是交织在一起的。

3.4.2 单帧滑动窗口与停止-等待协议



在**停止-等待协议**中，源站发送单个帧后必须等待确认，在目的站的回答到达源站之前，源站不能发送其他的**数据帧**。

从滑动窗口机制的角度看，**停止-等待协议相当于发送窗口的和接受窗口大小均为1的滑动窗口协议**。

超时重传： **超时重传**是指发送方在发送某个数据帧后就开启一个**超****时计时器**，**在一定时间内如果没有得到发送的数据帧的确认帧**，那么就重新发送该数据帧，知道发送成功为止。

超时计时器：每次发送一个帧就启动一个计时器

超时计时器设置的重传时间应当**比帧传输的平均RTT更长一些**。

在停止-等待协议中可能出现的差错：

(1) 数据帧丢失或检测到帧出错；

解决方法：超时重传；

注意：1、发送一个帧后，必须保留它的副本

2、数据帧和确认帧必须编号。

(2) **到达目的站的帧可能已遭到破坏**，接受站利用前面讨论过的差错检测技术检出后，简单地将该帧丢弃；

(3) (确认帧丢失) ACK丢失：**数据帧正确而确认帧被破坏，此时接收方已收到正确的数据帧，但发送方收不到确认帧。**

解决方法：超时重传，发送方会超时重传已被接受的数据帧，接收方收到同样的数据帧时会丢弃该帧，并重传一个该帧对应的确认帧；

发送的帧交替地用0和1来标识，肯定确认分别用ACK0和ACK1来表示，**收到的确认有误时，重传已发送的帧。**

(4) (确认帧迟到) ACK迟到：确认帧的到达时间超过了超时计时器

的时间。

解决方法：发送方会超时重传已被接受的数据帧，接收方收到同样的数据帧时会丢弃该帧，并重传一个该帧对应的确认帧。

为了对付上述**第二种可能出现的差错**，即到达目的站的帧可能已遭到破坏，源站装备了**计时器**。在一个帧发送之后，源站等待确认，**如果在计时器计满时仍未收到确认，那么在此发送相同的帧**。如此重复，直到该数据帧无错误地到达为止。

停止-等待协议算法的实现步骤：

在发送结点：

- (1) 从主机取一个数据帧，送交发送缓存；
- (2) $V(S) \leftarrow 0$ 。（发送状态变量 $V(S)$ 初始化）
- (3) $N(S) \leftarrow V(S)$ 。（将发送状态变量值写入写入数据帧中的发送序号 $N(S)$ ）
- (4) 将发送缓存中的数据帧发送出去。（这个数据帧的副本仍保留在发送缓存中）
- (5) 设置超时计时器。（选择适当的超时重传时间 t_{out} ）
- (6) 等待。（等待以下（7）和（8）这两个事件中最先出现的一个）
- (7) 收到确认帧 ACK_n 后，若 **$n=1-V(S)$ 则（已发送的数据帧被接收方确认）**从主机取一个新的数据帧，放入发送缓存。
 $V(S) \leftarrow [1-V(S)]$ ，转到（4）。（更新发送状态变量，变为下一个序号）否则，丢弃这个确认帧，转到（6）（表明已发送的数据帧未被接收方确认）
- (8) 若超时计时器时间到，则转到（4）（重传已发送的数据帧）

在接受结点：

- (1) $V(R) \leftarrow 0$ 。（接受状态变量初始化，其数值**等于欲接受的数据帧的发送序号**）

(2) 等待

(3) 收到一个数据帧时，检查有无产生传输差错（如用CRC）。若检查结果正确无误（**否则直接丢弃，转到（2）**），则执行后续算法。

(4) 若 $N(S) = V(R)$ ，则执行后续算法。（收到发送序号正确的数据帧），否则丢弃此数据帧，然后转到（7）。（**丢弃的帧就是重复帧**）

(5) 将收到的数据帧中的数据部分送交主机。

(6) $V(R) \leftarrow [1 - V(R)]$ 。（更新接受状态变量，准备接受下一个数据帧）

(7) 发送确认帧ACKn，并转到（2）。（ **$n = V(R)$ ，表明期望收到 $V(R)$** ）

由以上算法可知，对于停止-等待协议，由于**每发送一个数据帧就停止并等待**，因此**用1bit来编号**就已足够。

在停止-等待协议中，若**连续出现相同发送序号的数据帧**，表明发送端进行了**超时重传**。

若连续出**现相同序号的确认帧**，表明**接收端收到了重复帧**。

为了超时重传和判定重复帧的需要，发送方和接收方都须设置一个**帧缓冲区**，发送端发送完数据帧时，必须**在其发送缓存中保留此数据帧的副本**，这样才能在出差错时进行重传，**只有在收到对方发来的确认帧ACK时，方才清除此副本**。

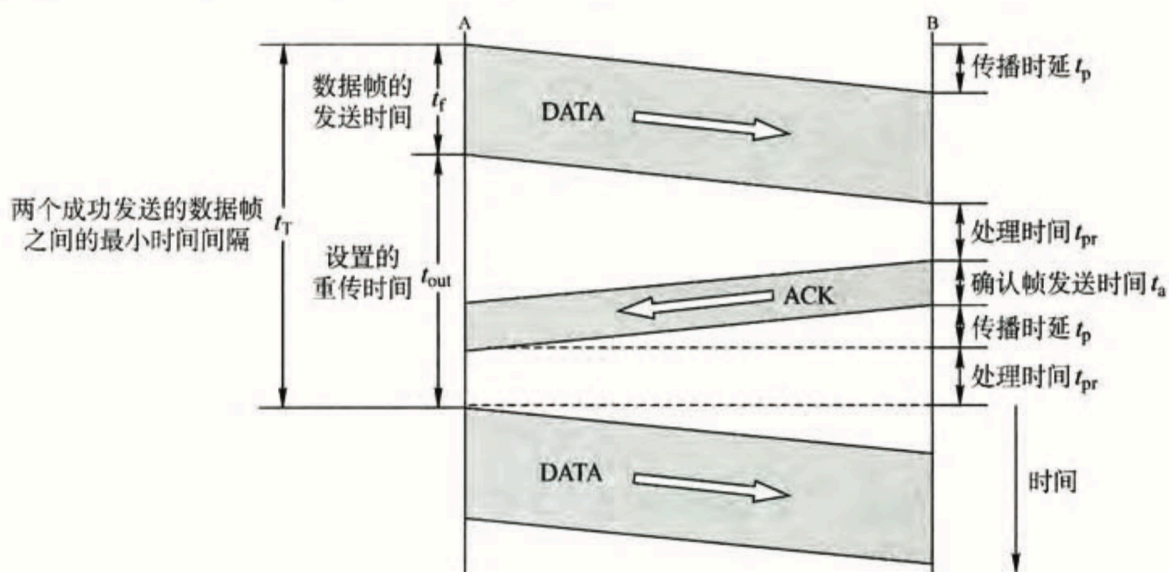


图 3.9 停止-等待协议中数据帧和确认帧的发送时间关系

停止等待协议的优点：简单

停止等待协议的缺点：停止-等待协议通信信道的利用率很低。

信道利用率：发送方在一个发送周期内，有效地发送数据所需要的时间占整个发送周期的比率。

即：信道利用率 = $(L/C) / T$

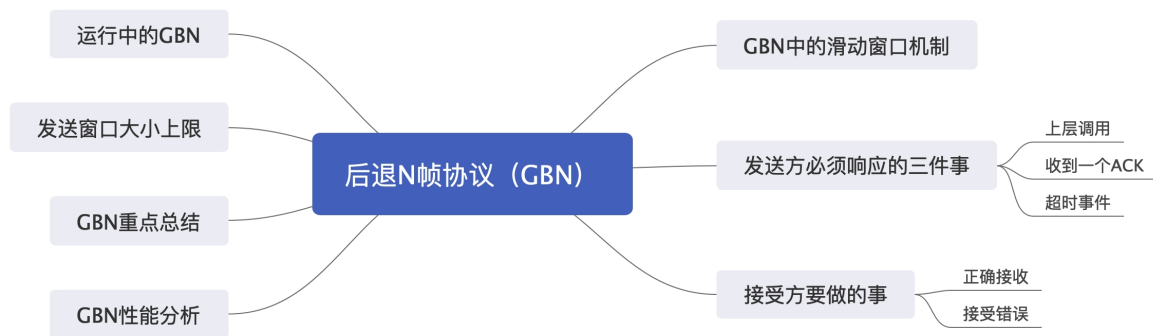
T：表示发送周期。从开始发送数据，到收到第一个确认帧为止。

L：表示T内发送L比特数据

C：表示发送方数据传输率

信道吞吐率 = 信道利用率 * 发送方的发送速率。

3.4.3 多帧滑动窗口与后退N帧协议（GBN）



在**后退N帧式ARQ**中，发送方**无须**在收到上一个帧的**ACK**后才能开始**发送下一帧**，而是**可以连续发送帧**。

GBN**发送方**必须响应的三件事

(1) 上层（即网络层）的调用：上层要发送数据时，发送方先检查发送串口是否已满。

如果**未**满，则产生一个帧并将其发送；

如果**已**满，发送方只需**将数据返回给上层**，暗示上层窗口已满，上层等一会在发送。

(2) 收到了一个ACK确认帧：

GBN协议中，对n号帧的确认采用**累积确认**的方式，标明接收方**已经收到了n号帧和它之前的全部帧**。

(3) 超时时间：协议的名字为后退N帧/回退N帧，来源于出现丢失和时延过长帧时发送方的行为。**如果出现超时，发送方重传所有已发送但未被确认的帧**。

GBN接收方要做的事：

(1)如果**正确收到n号帧**，并且**按序**，那么接受方为n帧**发送一个ACK**，并将该帧中的数据部分交付给上层。

(2)若不按序，则**丢弃不按序的帧**，并为**最近按序接收的帧重新发送一个ACK**。接收方无需缓存任何失序帧，只需要维护一个信息，即**下一个按序接收的帧序号**。

后退N帧协议大体执行过程：

(1) 当接收方**检测出失序的信息帧**后，要求**发送方重发最后一个正确接受的信息帧之后的所有未被确认的帧**；

(2) 或者**当发送方发送了N个帧后**，若发现**该N个帧的前一个帧在计时器超时后仍未返回其确认信息**，则该帧被判为出错或丢失，此时**发送方就不得不重传该出错帧及随后的N个帧**。

换句话说，**接收方只允许按顺序接受帧**。

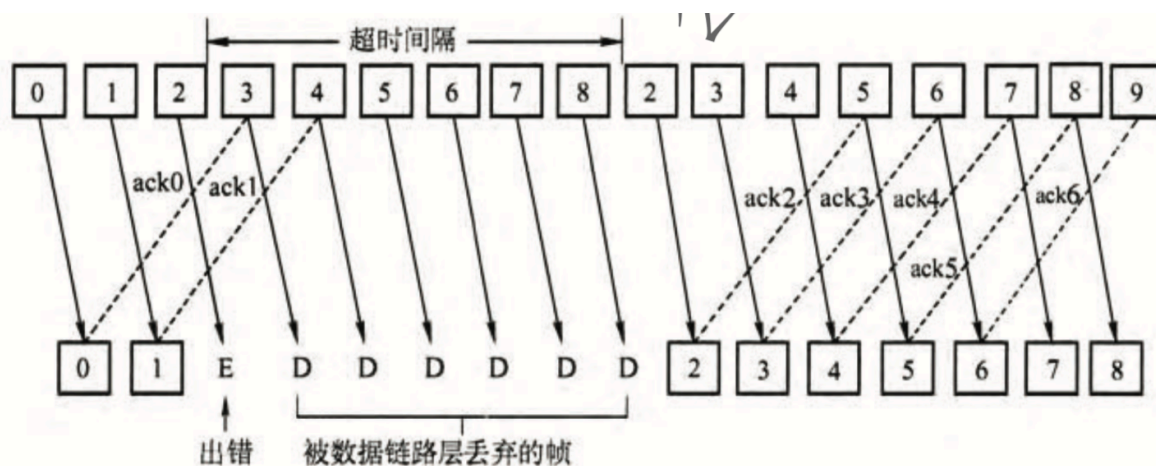


图 3.10 GBN 协议的工作原理：对出错数据帧的处理

如上图所示，源站向目的站发送0号帧后，可以继续发送后续的1号帧，2号帧等。

源站**每发送完一帧就要为该帧设置超时计时器**。

由于连续发送了许多帧，所以确认帧**必须要指明要对那一帧进行确认**。为了减少开销，而**可以在连续收到好几个正确的数据帧后，才对最后一个数据帧发确认信息**，或者可在自己有数据要发送时才将对以前正确收到的帧加以捎带确认。

这就是说，对某一数据帧的确认就表明该数据帧和此前所有的数据帧均已正确无误收到。

如上图所示，**ACKn表示对第n号帧的确认**，表示接收方已正确收到第n号帧及以前的所有帧，下一次期望收到第n+1号帧（也可能是第0号帧）。

接收端只按序接受数据帧。虽然在有差错的2号帧之后接着又收到了

正确的6个数据帧，但接受端都必须将这些帧丢弃。接收端虽然丢弃了这些不按序的无差错帧，但应**重复发送已发送的最后一个确认帧ACK1**，目的是为了**防止已发送的确认帧ACK1丢失**。

后退N帧协议的接受窗口为1，可以保证按序接受数据帧。若采用n比特对帧编号，则其**发送窗口的尺寸 W_T 应满足 $1 \leq W_T \leq 2^n - 1$** 。若发送窗口尺寸大于 $2^n - 1$ ，则会造成接收方无法分辨新帧和旧帧。

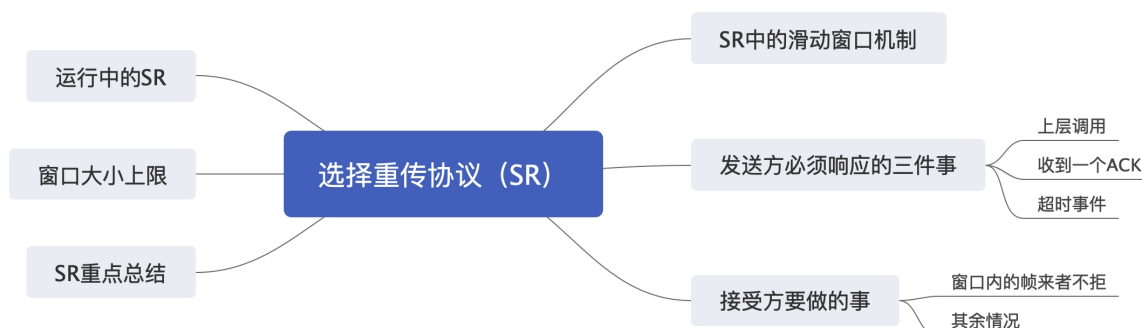
GBN协议重点总结：

- 1、累积确认（偶尔捎带确认）。
- 2、**接收方只按顺序接受帧，不按序无情丢弃。**
- 3、确认序列号最大的、按序到达的帧。
- 4、**发送窗口最大为 $2^n - 1$ ，接受窗口大小为1。**

GBN协议性能分析：

- (1) 因连续发送数据帧而提高了信道利用率
- (2) 在重传时必须把原来已经正确传送的数据帧重传，使传送效率降低。

3.4.4 多帧滑动窗口与选择重传协议（SR）



SR发送方必须响应的三件事：

(1) 上层的调用：从网络层收到数据后，SR发送方检查下一个可用于该帧的序号，如果序号位于发送窗口内，则发送数据帧率；否则就像GBN协议一样，要么将数据缓存，要么返回给上层之后在传输。

(2) 收到了一个ACK：

如果收到一个ACK，假如该帧序号在窗口内，则SR发送方将那个被确认的帧标记为已接收。如果该帧序号是窗口的下界（最左边的第一个窗口对应的序号），则窗口向前移动到具有**最小序号的未确认帧处**。如果窗口移动了并且有序号在窗口内的未发送帧，则发送这些帧。

(3) 超时事件

每个帧都有一个自己的定时器，一个超时事件发生后**只重产一个帧**。

SR接收方要做的事

(1) 窗口内的帧来者不拒

SR接受方将确认一个正确接收的帧而**不管是否按序**。**失序的帧将被缓存，并返回给发送方一个该帧的确认帧（收谁确认谁），直到所有帧（即序号跟更小的帧）皆被收到为止**，这时才可以将一批帧按序交付给上层，然后向前移动滑动窗口。

选择重传协议ARQ协议内容：

为进一步提高信道的利用率，可**设法只重传出现差错的数据帧或计时器超时的数据帧**。但此时必须**加大接受窗口**，以便**先收下发送序号不连续但仍处于接受窗口中的那些数据帧**。**等到所缺序号的数据帧收到后再一并送交主机**。

在选择重传协议中，**每个发送缓冲区对应一个计时器**，**当计时器超时**时，缓冲区的帧就会重传。

另外，该协议使用了比上述其他协议更有效的差错处理策略，即**一旦接收方怀疑帧出错**，就会**发一个否定帧NAL给发送方**，**要求发送方对NAK中指定的帧进行重传**。

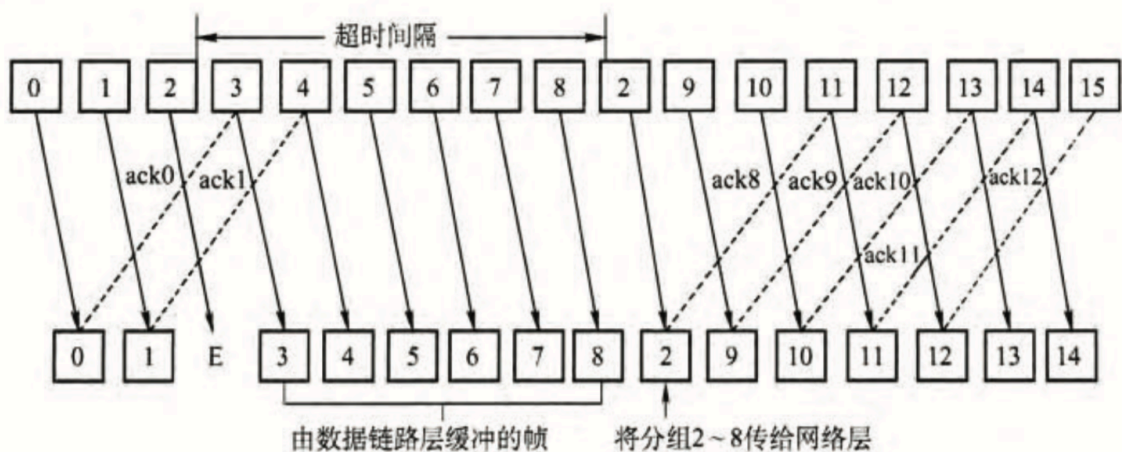


图 3.11 选择重传协议

选择重传协议的**接受窗口尺寸 W_R** 和**发送窗口尺寸 W_T** 都大于1，一次可以发送或接收多个帧。

若采用 **n 比特对帧编号**，为了保证接收方向前移动窗口后，**新窗口序号与旧窗口序号没有重叠部分**，需要满足条件：**接受窗口 W_R +发送窗口 $W_T \leq 2^n$** 。

假定仍然采用累计确认的方法，并且**接受窗口 W_R** 显然不应超过**发送窗口 W_T** （否则无意义），那么**接收窗口尺寸不应超过序号范围的一半**，即 **$W_R \leq 2^{(n-1)}$** 。接受窗口为最大值时， **$W_{Tmax} = W_{Rmax} = 2^{(n-1)}$** 。

需要注意的是，一般情况下，在SR协议中，**接受窗口的大小和发送窗口的大小是相同的**。

SR协议的重点总结：

- 1、对数据帧逐一确认，收一个确认一个
- 2、只重传出错帧
- 3、接收方有缓存

补充几个概念：

(1) **信道的效率**：也称**信道利用率**。可从不同的角度来定义信道的效率，这里给出一个从时间角度的定义：**信道效率是对发送方而言**

的，是指发送方在一个发送周期的时间内，有效地发送数据所需要的时间占整个发送周期的比率。

例如：在发送方从开始发送数据到收到第一个确认帧为止，称为一个发送周期，设为 T ，发送方在这个周期内共发送 L 比特的数据，发送方的数据传输率为 C ，则发送方用于发送有效数据的时间为 L/C ，在这种情况下，信道的利用率为 $(L/C) / T$ 。

从上面的讨论可以发现，求信道的利用率主要是求周期时间 T 和有效数据发送时间 L/C 。

信道吞吐率=信道利用率*发送方的发送速率。